# Project Prometheus Workplan v0.110-v0.119: The Causal Isomorphic Agent for Strategic God-Games

## Executive Summary

This document outlines the comprehensive workplan for the v0.11x development cycle of the Project Prometheus technology demonstrator. This cycle represents a strategic inflection point, moving beyond the foundational self-improvement capabilities established in the v0.9x and v1.0 series to pursue a more profound synthesis of intelligence. The central goal is the creation of a **Causal Isomorphic Agent**: a system that learns and improves not merely by optimizing for a win condition, but by actively seeking to build a high-fidelity, causal, and multi-layered internal model of its environment that is *isomorphic* to the external reality of the *0 A.D.* strategic god-game.[1]

This workplan systematically integrates the foundational principles of two seminal thinkers. From I.J. Good, it operationalizes the mechanisms of an "intelligence explosion": Recursive Self-Improvement (RSI), a "causal calculus" for reasoning, and an internal governance system for strategic stability.[1] From Douglas R. Hofstadter, it architects the cognitive structures that give rise to meaning and creativity: the "Strange Loop" of self-reference, "isomorphism" as the bedrock of meaning, and "analogy" as the engine of creative problem-solving.[1]

The 10-version cycle is structured into three distinct phases, each building upon the last to cultivate progressively more sophisticated cognitive capabilities:

1. **Phase I (v0.110-v0.112): Grounding the Self** - This initial phase is dedicated to establishing the agent's perceptual and conceptual foundation. It involves building a high-fidelity, isomorphic world-model of the *0 A.D.* environment and developing the capacity to abstract low-level action sequences into symbolic "chunks" of strategic knowledge.
2. **Phase II (v0.113-v0.115): The Emergence of Causal Reason** - Building on a stable world-model, this phase implements the core of causal reasoning. The agent will move beyond correlation to understand causation, enabling a self-referential critique loop

where it can analyze the reasons for its failures and propose targeted self-corrections.
3. **Phase III (v0.116-v0.119): The Creative Leap** - The final phase aims to unlock genuine strategic creativity. It introduces analogical reasoning, allowing the agent to solve novel problems by transferring knowledge across disparate contexts. The RSI process is then turned inward, tasking the agent with optimizing its own computational efficiency, culminating in a comprehensive system test and a final act of metacognitive self-analysis.

All development will be grounded in the practicalities of our target compute platforms. The powerful GPUs available on Google Colab will be leveraged for training, evolutionary search, and other computationally intensive tasks.[3] Optimized inference agents will be deployed to the NVIDIA Jetson Orin Nano, making real-time performance on resource-constrained edge hardware a primary driver of self-improvement.[6] The successful completion of this cycle will yield an agent that not only plays a complex game at a high level but demonstrates the emergent properties of causal understanding, strategic creativity, and hardware-aware self-optimization that are the hallmarks of true general intelligence.

# Section 1: Architectural Synthesis for v0.11x - The Causal Isomorphic Agent

This section establishes the guiding architectural philosophy for the entire v0.11x cycle. It moves beyond a simple feature list to articulate a unified vision where Goodian and Hofstadterian concepts are not merely added but are synthesized into a coherent, self-reinforcing whole.

## Core Thesis: Synthesizing the Engine and the Nature of Intelligence

The v0.11x cycle is predicated on the insight that the frameworks of I.J. Good and Douglas R. Hofstadter describe two orthogonal, complementary aspects of intelligence. Good's framework provides the blueprint for the *engine* of intelligence growth—the "how" of recursive self-improvement and the intelligence explosion.[1] Hofstadter's framework, conversely, describes the *nature* of the intelligence that emerges—the "what" of meaning, self-awareness, and creativity.[1] An intelligence explosion is most potent and generalizable when the intelligence being amplified is one that seeks meaning through isomorphism and achieves creative breakthroughs through analogy. A purely Goodian agent risks optimizing itself into a brittle, "alien" super-performer, while a purely Hofstadterian agent risks becoming a passive "philosopher," capable of deep understanding but lacking the relentless drive for improvement. This workplan synthesizes these two views: it proposes using Good's RSI engine to relentlessly improve an agent's ability to perform Hofstadterian cognition.

## The Causal Isomorphic Agent (CIA) Paradigm

To formalize this synthesis, the target architecture is defined as the **Causal Isomorphic Agent (CIA)**. The CIA's primary utility function is a composite of two equally weighted objectives: (1) achieving external goals within its environment (e.g., winning a match in *0 A.D.*), and (2) minimizing the measured "distance" between its internal world-model and the true causal structure of the external environment. This dual objective is critical; it forces the agent to improve not just its performance but its *understanding*. Success is no longer just about finding a winning policy but about building a correct, predictive, and causally-sound model of the world, from which winning policies naturally emerge. This approach is designed to produce a more robust and generalizable form of intelligence than that of a pure reward-maximizer.

## Mapping Theory to Prometheus Architecture

The CIA paradigm is realized by mapping the synthesized theoretical concepts onto the established three-layer Prometheus architecture (Capability, Metacognitive, Safety).

- **Good's "Subassemblies" & Hofstadter's "Chunking" -> Causal Agentic Mesh (CAM):** Good's theory of "assemblies and subassemblies" posits that thoughts are composed of reverberating, semi-persistent neural circuits that can be shared and combined to form more complex ideas.[1] Hofstadter's concept of "chunking" describes the cognitive process of abstracting recurring low-level patterns into singular, higher-level symbols, which is fundamental to building hierarchical understanding.[1] The v0.11x architecture implements this synergy within the **Causal Agentic Mesh (CAM)**.[1] In this evolution of the architecture, the CAM is not a collection of static, monolithic experts. Instead, it is a dynamic substrate composed of composable "expert circuits," each representing a specific strategic concept or "chunk" (e.g., "Economic Boom," "Pincer Attack," "Naval Blockade"). The Self-Modification Module (SMM) will be explicitly tasked with discovering these recurring patterns in gameplay and instantiating them as new, named expert circuits. This directly implements Hofstadterian chunking as the mechanism for building the Goodian subassemblies of strategic thought.
- **Good's "Causal Calculus" & Hofstadter's "Jumping Out of the System" -> Causal

**Attention & CRLS:** Good argued that true intelligence requires a "causal calculus"—a formal method for reasoning about the outcomes of interventions and counterfactuals.[1] This stands in contrast to the purely correlational reasoning of standard machine learning models.[1] Hofstadter describes creative leaps and the resolution of paradox as "jumping out of the system," an act of meta-level critique or analogical insight that transcends the current formal rules.[1] The v0.11x plan engineers this capability by evolving the agent's reasoning from correlational to causal. This is achieved through two key components: a **"Causal Attention Head"** wrapper and the **Causal Reinforcement Learning from Self-Correction (CRLS)** loop.[1] The Causal Attention Head forces the agent to evaluate actions by asking counterfactual questions ("What would have happened if I had attacked *here* instead of *there*?"), a practical implementation of a causal calculus.[10] The CRLS loop then uses the answers to these questions to generate a symbolic, causal critique of the agent's performance, which becomes the input for the next cycle of self-modification. This self-referential critique is the engine of Hofstadter's Strange Loop.

- **Good's "Centrencephalic System" -> The Modern Centrencephalic System (MCS):** To manage the stability of his proposed "ultraparallel neural net," Good drew inspiration from Wilder Penfield's neuroscience, proposing an internal governor or "centrencephalic system" to maintain high-level focus and coherence.[1] This concept maps directly to the project's existing **Modern Centrencephalic System (MCS)**.[1] During this development cycle, the MCS will evolve from its current role as a simple safety backstop into a sophisticated, high-level, learned gating network. Its primary function will be to dynamically allocate the system's finite computational resources (specifically, GPU cycles on the Jetson Orin Nano) to the most strategically relevant "subassemblies" (expert circuits) within the CAM based on the global game state. This provides strategic coherence and prevents the agent from becoming paralyzed by tactical calculations, directly implementing Good's vision of an internal governor for a complex cognitive architecture.

# Section 2: Detailed Workplan: The v0.110 - v0.119 Development Cycle

This section provides the granular, version-by-version roadmap for the development cycle. Each version represents a discrete, verifiable step in the construction of the Causal Isomorphic Agent, with clear objectives, technical specifications, and success criteria.

### Table 1: v0.110-v0.119 Feature and Verification Matrix

| Version | Phase | Core Concept | Key Development Task | Primary Verification Metric |
|---|---|---|---|---|
| **v0.110** | I | Hofstadter: Isomorphism | Implement a high-fidelity internal world-model of the *0 A.D.* game state using the external RL interface. | IEE can measure and report a quantifiable "Isomorphism Fidelity Score" > 99.9% against the ground-truth game state. |
| **v0.111** | I | Good/Hofstadter: Subassemblies/Chunking | SMM identifies recurrent action sequences and symbolizes them as dynamic, composable "expert circuits" within the CAM. | Agent demonstrates use of at least 5 named "strategic chunks" (e.g., execute_cavalry_flank) that improve performance over monolithic policy. |
| **v0.112** | I | Good: Centrencephalic System | Implement the MCS as a learned gating network that allocates GPU resources to CAM expert circuits based on game state. | Measurable reduction in inference latency and GPU power draw on Jetson Orin Nano by >15% with no loss in performance. |

| | | | | |
|---|---|---|---|---|
| **v0.113** | II | Good: Causal Calculus | Develop a "Causal Attention Head" wrapper using prompt engineering to force counterfactual tactical evaluation. | Agent can correctly answer "what if" questions about past battles with >90% accuracy against simulated ground truth. |
| **v0.114** | II | Hofstadter: Strange Loop | Integrate the CRLS cycle where the IEE's causal critique of a game is the SMM's input for modifying CAM circuit weights. | Agent demonstrates ability to self-correct a losing strategy (e.g., "stop building spearmen against archers") within 3 generations based on a symbolic critique. |
| **v0.115** | II | Hofstadter: Gödelian Flag | IEE flags novel, high-risk strategies generated by the SMM as "UNDECIDABLE," halting execution and triggering HITL protocol. | System correctly halts and requests human approval when presented with a known "game-breaking" but potentially winning exploit. |

| | | | | |
|---|---|---|---|---|
| **v0.116** | III | Hofstadter: Analogy | SMM's "Analogical Search Engine" abstracts a problem's "conceptual skeleton" and transfers a solution from a different context. | Agent, when facing a "turtling" defense, successfully applies a strategy learned from a completely different "economic blockade" scenario. |
| **v0.117** | III | Good: RSI on Efficiency | IEE profiles GPU/CPU usage and latency on Jetson Orin Nano. SMM self-modifies code to improve computational efficiency. | Agent autonomously improves its own frames-per-second (FPS) performance by >20% through self-generated code optimizations (e.g., quantization, batching). |
| **v0.118** | III | GEB: Six-Part Ricercar | Integrated system test in a custom *0 A.D.* scenario requiring all v0.11x features to achieve victory. | Agent wins the "Ricercar" scenario against the benchmark Petra AI on "Very Hard" difficulty. |
| **v0.119** | III | Good/Hofstadter: | Agent analyzes its own v0.11x | Generated workplan is |

| | | | | |
|---|---|---|---|---|
| | | Meta-Cognition | development history and generates a data-driven workplan proposal for the v0.12x cycle. | coherent, technically plausible, and identifies at least one valid, non-obvious area for future improvement. |

---

## Phase I: Grounding the Self - Isomorphism and Subassemblies (v0.110 - v0.112)

This initial phase focuses on constructing the agent's perceptual bedrock—its internal model of the world—and the basic symbolic units of its strategic thought. The goal is to create an agent that can see the world clearly and think in concepts before it learns to reason deeply.

### v0.110: Isomorphic Grounding in the 0 A.D. World-State

- **Objective:** To establish a robust, high-fidelity, and continuously updated internal representation of the *0 A.D.* game world that will serve as the foundation for all subsequent reasoning and learning.
- **Theoretical Foundation:** This task is a direct implementation of Hofstadter's principle that meaning arises from a structure-preserving map—an isomorphism—between an internal symbolic system and an external domain of reality.[1] This moves beyond the simple state-vector representations common in reinforcement learning to the explicit architectural goal of creating a "perfect mirror" of the game world, where the pursuit of meaning (i.e., a more perfect mirror) becomes a quantifiable objective.[12]
- **Architectural Modifications:**
  - **Capability Layer:** A new "State Ingestion" module will be developed. This module's sole responsibility is to manage communication with the *0 A.D.* external Reinforcement Learning (RL) HTTP interface.[1] It will be engineered to continuously poll the game's JSON state endpoint at a high frequency, handle asynchronous data streams, and parse the raw state information.
  - **Metacognitive Layer (IEE):** The Introspection and Evaluation Engine (IEE) will be enhanced with an "Isomorphism Fidelity Verifier." On every game tick, this module will perform a deep comparison between the agent's internal knowledge base

representation of the game state and the ground-truth JSON data received from the game engine.

- **Implementation Details:**
  - The agent's internal world-state will be represented using a formal knowledge graph structure, likely using a Python library that supports Property Graphs. This is a critical choice, as a graph structure allows for the explicit representation of entities (nodes) and their relationships (edges), which is essential for measuring structural correspondence—the core of isomorphism—in a way that a flat dictionary or vector cannot.[1]
  - The State Ingestion module will parse the asynchronous JSON data provided by the *0 A.D.* RL interface, which details all units, buildings, resources, and their properties, and translate this hierarchical data into the graph-based world model.[1]
  - Initial development and testing will be conducted in a Google Colab environment. This will involve running a headless, containerized instance of *0 A.D.* and using the provided Python client to interact with the RL interface, allowing for rapid iteration on the data parsing and graph construction logic.[1]
- **Verification Protocol:** The primary verification metric is the "Isomorphism Fidelity Score." This score will be calculated by the IEE as the percentage of entities, properties, and relationships in the agent's internal graph model that perfectly match the ground truth from the game engine's state dump. The strict success criterion for this version is achieving a sustained fidelity score greater than 99.9% under the stress of a dynamic, multi-unit game scenario.

### v0.111: Goodian Subassemblies as Strategic Chunks

- **Objective:** To begin building a library of high-level, symbolic strategic concepts ("chunks") by abstracting them from low-level action sequences. These chunks will form the basic vocabulary of the agent's strategic thought.
- **Theoretical Foundation:** This version represents a direct synthesis of two core ideas. The first is Hofstadter's concept of "chunking," the cognitive mechanism for building levels of abstraction by grouping lower-level patterns into higher-level symbols.[1] The second is I.J. Good's "subassembly theory," which posits that thoughts and concepts are physically embodied as reusable, semi-persistent circuits of neurons that can be combined to form more complex thoughts.[1] In this architecture, a strategic chunk *is* a Goodian subassembly.
- **Architectural Modifications:**
  - **Metacognitive Layer (SMM):** The Self-Modification Module (SMM) will be upgraded with a "Pattern Miner" module. This module will be designed to analyze the complete action history of winning game traces. Initially, these traces will be sourced from a database of human expert replays; in later versions, they will come from the agent's

own successful games. The Pattern Miner's function is to identify frequently recurring, effective sequences of actions (e.g., the specific sequence of build_barracks -> train_swordsman * 5 -> attack_enemy_farm).
    - **Causal Agentic Mesh (CAM):** When the SMM's Pattern Miner identifies and validates a new strategic chunk, it will instantiate a new, dynamic "expert circuit" within the CAM.[1] This circuit is not a static part of the model but a composable cognitive resource. It can be implemented as either a small, specialized policy network fine-tuned for that specific task or as a sophisticated, reusable prompt chain for the core LLM. This circuit can then be invoked by a single symbolic command (e.g., execute_infantry_rush), allowing the agent to reason at a higher level of strategic abstraction.
- **Implementation Details:** The Pattern Miner will be implemented using established sequence mining algorithms (e.g., PrefixSpan). The CAM will be architected as a Mixture-of-Experts (MoE) model, with the crucial distinction that the "experts" are not fixed at training time but can be dynamically created, versioned, and composed by the SMM during the RSI loop.[1]
- **Verification Protocol:** The agent must successfully identify, create, and name at least five distinct, strategically meaningful chunks from the provided replay data. In subsequent gameplay tests, the agent must demonstrate the ability to invoke these chunks symbolically as part of its planning process. The use of these chunks must lead to a measurable improvement in performance (e.g., a statistically significant increase in win rate or a decrease in time-to-victory) compared to the monolithic policy of v0.110.

### v0.112: The Centrencephalic Governor for Strategic Focus

- **Objective:** To implement a meta-level controller that dynamically manages the agent's "attention" and computational resources, preventing it from becoming paralyzed by an overwhelming number of tactical possibilities and enabling it to maintain coherent, long-term strategic focus.
- **Theoretical Foundation:** This is a direct implementation of I.J. Good's proposal for a "centrencephalic system," an internal governor inspired by the work of neurosurgeon Wilder Penfield. Good envisioned this system as essential for maintaining stability and providing high-level control in his proposed "ultraparallel neural net".[1] For our agent, the "ultraparallel net" is the collection of all possible strategies and subassemblies within the CAM, and "focus" is the efficient allocation of finite computational resources.
- **Architectural Modifications:**
    - **Safety Substrate (MCS):** The Modern Centrencephalic System (MCS) [1] will be promoted from its role as a simple safety checker to a dynamic, learned resource allocator. It will function as the top-level gating network for the entire CAM, effectively deciding which "thoughts" (expert circuits) are worth "thinking" at any

given moment.
    - **IEE:** The IEE will be extended with a "Performance Profiling" module. This module will be instrumented to profile the real-world computational cost (inference latency, GPU memory allocation, power draw) of each CAM expert circuit during execution on the NVIDIA Jetson Orin Nano target hardware.
- **Implementation Details:** The MCS will be implemented as a small, separate neural network trained via meta-learning. Its input will be a compressed, high-level representation of the global game state (derived from the Isomorphic World-Model). Its output will be a probability distribution (a set of gating weights) over all available expert circuits in the CAM. This distribution determines which circuits receive the most computational budget for the next planning cycle. This architecture directly addresses the critical need for high-performance, energy-efficient inference on resource-constrained edge devices like the Jetson Orin Nano.[7] This creates an internal "economy of thought," where computational cycles are a scarce resource to be allocated strategically.
- **Verification Protocol:** When the v0.112 agent is deployed on the Jetson Orin Nano, it must demonstrate a greater than 15% reduction in average decision latency (time per game tick) and a corresponding reduction in average GPU power consumption compared to the v0.111 agent (which activates all circuits equally). This performance gain must be achieved with no statistically significant drop in win rate against the benchmark AI, providing a concrete, measurable demonstration of improved strategic "focus" and computational efficiency.

## Phase II: The Emergence of Causal Reason - Causal Calculus and Strange Loops (v0.113 - v0.115)

This phase transitions the agent from a pattern-matching strategist, which understands *what* works, to a causal reasoner, which begins to understand *why* it works. This is the critical step toward genuine intelligence and robust self-improvement.

### v0.113: Causal Attention for Tactical Decision-Making

- **Objective:** To imbue the agent with a primitive form of causal reasoning, enabling it to evaluate tactical decisions based on simulated counterfactual outcomes rather than relying solely on correlational patterns learned from past data.
- **Theoretical Foundation:** This is a practical application of I.J. Good's call for a "causal calculus," which he identified as a necessary component of an ultraintelligent machine.[1] It

recognizes that true intelligence requires the ability to reason about the effects of interventions ("what if I do X?"). This moves the agent's reasoning beyond the purely correlational nature of standard Transformer self-attention mechanisms.[1]

- **Architectural Modifications:**
  - **Capability Layer:** A "Causal Attention Head" wrapper will be implemented around the core LLM engine.[1] It is crucial to note that this is not a modification of the foundation model's internal architecture, which is infeasible. Rather, it is a sophisticated prompt-engineering and control module that orchestrates the LLM's existing capabilities to perform causal inference.
- **Implementation Details:** When a critical tactical decision is required (e.g., "Should my group of spearmen attack the approaching enemy cavalry?"), the Causal Attention Head will intercept the query. Instead of asking the LLM for a single, direct action, it will generate a batch of counterfactual prompts based on the available action space. For example:
  1. "Given the current game state, simulate the most likely outcome of the next 15 seconds if the spearmen attack the cavalry."
  2. "Given the current game state, simulate the most likely outcome of the next 15 seconds if the spearmen hold their defensive position."
  3. "Given the current game state, simulate the most likely outcome of the next 15 seconds if the spearmen retreat towards the main base."
     The LLM's generated predictions for each scenario will be parsed and evaluated by a utility function within the wrapper (e.g., based on projected unit losses). The action corresponding to the best simulated counterfactual outcome will be selected and executed. This approach leverages the powerful world-simulation capabilities latent within modern LLMs to approximate a causal intervention, a core principle of Causal Reinforcement Learning (CRL).11
- **Verification Protocol:** A new verification suite of "tactical puzzles" will be created. The agent will be presented with saved game states from historical battles and asked specific counterfactual questions (e.g., "In this replay, Player A lost the battle. Would Player A have won if they had built three archers instead of three cavalry units at timestamp X?"). The agent's answers will be compared against ground-truth simulations run in the game engine. Success is defined as achieving greater than 90% accuracy on this puzzle suite.

### v0.114: The Self-Correction Strange Loop

- **Objective:** To close the full metacognitive feedback loop, enabling the agent to reason about its own strategic failures in causal terms and implement targeted, corrective self-modifications. This transforms the agent from a system that is merely improved by external feedback to one that truly learns from its own mistakes.
- **Theoretical Foundation:** This version operationalizes Hofstadter's "Strange Loop," the

central concept of *Gödel, Escher, Bach*. A Strange Loop occurs when a system can perceive, represent, and act upon itself, creating a "tangled hierarchy" where the distinction between levels of cause and effect becomes fluid.[1] This workplan implements the loop as follows: **Act (play game) -> Observe Outcome (lose) -> Generate Causal Critique (why did I lose?) -> Reason on Critique (what strategy needs to change?) -> Modify Self (rewrite the strategy)**.
- **Architectural Modifications:**
  - **IEE:** The "Critique Generator," first conceptualized in the v0.9x series, will be fully implemented.[1] After a loss, the IEE will use the Causal Attention Head (from v0.113) to analyze the game replay. Its goal is to generate a high-level, natural-language causal attribution for the defeat (e.g., "Defeat was caused by a critical resource deficit in the mid-game, stemming from a failure to scout the western flank, which allowed an undefended resource line to be raided by enemy cavalry.").
  - **SMM:** The SMM will be fundamentally re-architected to accept this symbolic, causal critique as its primary input for proposing modifications. It will use the critique to perform causal credit assignment, identifying which CAM subassemblies (strategic chunks) were causally responsible for the failure. It will then propose specific modifications, such as increasing the activation weight of the "Scouting Patrol" chunk in the early-game policy or creating a new "Resource Defense" expert circuit.
- **Implementation Details:** This architecture implements the Causal Reinforcement Learning from Self-Correction (CRLS) loop.[1] The "reward signal" is no longer a simple scalar (+1 for a win, -1 for a loss) but a rich, symbolic, causal explanation of performance. This provides a much more targeted and data-efficient learning signal than traditional RL, which often struggles with sparse rewards and credit assignment in complex games.[22]
- **Verification Protocol:** The agent will be initialized with a policy containing a known, deliberate strategic flaw (e.g., a strong bias toward producing only one unit type, regardless of the enemy's composition). The system must demonstrate that, after a small number of losses (e.g., fewer than 5), it can correctly identify the causal flaw via the IEE's critique and successfully modify its own strategy via the SMM to overcome the flaw and begin winning.

### v0.115: The Gödelian Flag for Strategic Novelty

- **Objective:** To implement a robust safety protocol that can manage the inherent risks of a creatively self-improving agent, specifically its potential to discover novel but potentially misaligned, unsafe, or "game-breaking" strategies.
- **Theoretical Foundation:** This is a direct application of Gödel's Incompleteness Theorems, as interpreted by Hofstadter in the context of intelligent systems.[1] The agent's objective function (e.g., "win the game within the rules") acts as its formal system of axioms. A sufficiently creative SMM, through its evolutionary processes, will inevitably

generate a novel strategy (a "theorem") whose alignment with the system's core principles is formally undecidable from within the IEE's own analytical framework. The goal is not to prevent this, but to detect and manage it.

- **Architectural Modifications:**
  - **IEE:** The "Consistency Checker" module, first designed in the v0.9x plan, will be implemented.[1] When the SMM proposes a new strategy or a modification to an existing one, the IEE will classify it into one of three categories: PROVABLY_ALIGNED (e.g., a simple efficiency improvement to a known strategy), PROVABLY_MISALIGNED (e.g., a change that causes it to attack its own units, violating a core principle), or the crucial UNDECIDABLE.
  - **Safety Substrate:** An UNDECIDABLE flag from the IEE will act as a circuit-breaker, triggering a hard-coded "Jump Out of the System" protocol within the Safety Substrate. This protocol immediately halts the RSI loop for the proposed modification and escalates the novel strategy, along with the IEE's analysis, to a human operator for review and final judgment. This will be facilitated by a simple web interface, potentially using a framework like Gradio.[24]
- **Implementation Details:** The "undecidability" check will necessarily be heuristic-based. It will be triggered by proposed strategies that exhibit a combination of high novelty (low structural similarity to any known strategy in the agent's "gene bank"), high risk (involve sacrificing a large percentage of the agent's resources for a potential long-term gain), or appear to exploit potential ambiguities or loopholes in the game engine's physics or rules.
- **Verification Protocol:** A "red team" test will be conducted. A known but obscure and powerful exploit in the *0 A.D.* game engine will be seeded into the SMM's evolutionary search space. The system must demonstrate the full safety pipeline: (1) The SMM must be creative enough to independently discover and formulate the exploit strategy. (2) The IEE must correctly identify this novel, high-risk strategy and flag it as UNDECIDABLE. (3) The Safety Substrate must successfully halt the autonomous loop and present the strategy to the human operator for review.

## Phase III: The Creative Leap - Analogy and the Intelligence Explosion (v0.116 - v0.119)

This final phase aims to unlock true creative problem-solving by moving beyond logical deduction and causal reasoning into the realm of analogy. It culminates in a demonstration of a tangible, hardware-aware intelligence explosion.

### v0.116: The Analogical Search Engine for Creative Strategy

- **Objective:** To enable the agent to solve novel, unforeseen problems by abstracting their core structure and applying solutions from structurally similar but superficially different past experiences.
- **Theoretical Foundation:** This is a direct implementation of Hofstadter's central thesis that analogy is the "core of cognition" and the primary mechanism for creative, "out-of-the-box" thinking that allows intelligent systems to escape the rigid confines of their own formal logic.[1] This serves as the engineered solution to the Gödelian limitations and undecidability encountered in v0.115.
- **Architectural Modifications:**
  - **SMM:** A new **"Analogical Search Engine"** will be constructed within the SMM. This engine will consist of two primary sub-modules, inspired by the architecture of Hofstadter's Copycat project [27]:
    1. **Conceptual Skeleton Extractor:** This module takes a problem state (e.g., a losing game board configuration) and generates a high-level, abstract graph representation of its core causal and relational structure—its "conceptual skeleton." This process involves stripping away all domain-specific surface details (e.g., unit types, map location) to reveal the underlying pattern. For example, "My single fortified base is being starved of resources by an enemy controlling all external resource nodes" becomes an abstract graph: -> ->.
    2. **Cross-Domain Mapper:** When faced with a novel problem for which no existing strategy is effective, this module is activated. It searches the IEE's entire historical "gene bank" of past game situations, looking for a scenario with an isomorphic conceptual skeleton, regardless of whether that past scenario was from the early-game, late-game, involved different units, or even occurred on a different map. Upon finding a match, it uses the structural mapping to "translate" the successful solution strategy from the old, analogous context into a novel strategy for the current problem.
- **Implementation Details:** The creation, storage, and matching of "conceptual skeletons" is the core technical challenge. This process is computationally expensive and will be offloaded to run on the powerful GPUs available in the Google Colab environment. The search for isomorphic graphs is a known hard problem, and approximate methods will be necessary.
- **Verification Protocol:** A custom scenario will be designed to test this capability directly. The agent will be placed in a late-game "siege" scenario where it is defensively "turtling" and has never encountered this specific situation before. It will, however, have access to its memory of successful early-game "economic rush" and "blockade" scenarios. The agent must successfully identify the abstract structural analogy between being economically choked in the early game and being militarily besieged in the late game. It must then adapt an early-game resource acquisition or blockade-breaking strategy into a

successful late-game military "sally" or "breakout" maneuver to win the scenario.

### v0.117: GPU-Aware Recursive Self-Improvement

- **Objective:** To make the agent's own computational efficiency a primary target for its RSI loop, demonstrating a tangible and measurable "intelligence explosion" in terms of strategic performance per watt.
- **Theoretical Foundation:** This is the ultimate practical application of I.J. Good's core concept of RSI.[1] If an ultraintelligent machine can improve its own intelligence, it must also be able to improve the efficiency of the physical substrate on which that intelligence operates. This turns the limitations of the hardware into a driver for improvement.[34]
- **Architectural Modifications:**
  - **IEE:** The IEE's "Performance Profiling" module will be fully instrumented with NVIDIA's profiling tools to monitor real-time performance metrics on the Jetson Orin Nano. This includes GPU/CPU utilization, memory bandwidth, inference latency (in tokens/sec or decisions/sec), and total power draw from the module. These hardware metrics will be added as explicit terms in the agent's multi-objective utility function.
  - **SMM:** The SMM's action space will be significantly expanded. It will now be able to propose modifications not just to the agent's strategic logic (the CAM circuits) but to its own underlying implementation code. Its goal becomes a complex multi-objective optimization problem: maximize win rate AND minimize resource consumption (latency, power).
- **Implementation Details:** The agent will be given access to a sandboxed toolchain that includes libraries like NVIDIA's TensorRT-LLM [35] and knowledge of optimization techniques such as model quantization and kernel fusion. A successful self-modification in this version might involve the SMM proposing a code change to rewrite a frequently used but non-critical expert circuit from FP16 to INT8 precision. The IEE would then have to verify that this change achieves the desired improvement in FPS on the Jetson Orin Nano without causing an unacceptable degradation in the agent's win rate. This directly leverages the specific hardware capabilities of the Jetson Orin architecture, such as its dedicated Tensor Cores for INT8 acceleration.[7]
- **Verification Protocol:** The agent will be deployed on a Jetson Orin Nano and tasked with playing a series of games. It must autonomously generate and commit a code modification to its own software stack that increases its decision-making throughput (measured in game-states evaluated per second) by at least 20%. This improvement must be validated while maintaining its win rate against a fixed benchmark AI, demonstrating a concrete, hardware-aware self-improvement.

### v0.118: Integrated System Test - The Six-Part Ricercar

- **Objective:** A comprehensive, capstone system test designed to verify that all the individual Goodian and Hofstadterian components developed throughout the v0.11x cycle can operate in concert to solve a complex, multi-faceted strategic problem in real-time.
- **Theoretical Foundation:** This test is named in honor of the culminating, unifying piece in J.S. Bach's *Musical Offering*, which weaves multiple complex themes together, and serves as the successor to the integrated system test from the v0.9x series.[1] It represents the final synthesis of all the project's themes.
- **Implementation Details:** A custom *0 A.D.* scenario, titled "The Ricercar," will be designed specifically to be unsolvable by any single, monolithic strategy. Victory will require the successful, integrated application of all the key capabilities developed in this cycle:
  1. **Isomorphic Fidelity:** The scenario will involve hidden units and deception, requiring the agent to maintain an accurate internal model despite incomplete information.
  2. **Subassembly Use:** Victory requires executing complex, multi-step build orders and precisely timed, multi-pronged attack plans, invoking the "strategic chunks" from the CAM.
  3. **Causal Reasoning:** A key mid-game battle can only be won by making a counter-intuitive tactical sacrifice, a decision that can only be reached through counterfactual evaluation.
  4. **Strange Loop Correction:** The agent will be baited into an initial, flawed strategy that leads to an early setback, forcing it to use the CRLS loop to diagnose the failure and adapt its overall plan.
  5. **Analogical Leap:** The final phase of the scenario will present a novel defensive puzzle that can only be solved by the agent making an analogical connection to an offensive strategy it learned in a different context.
  6. **Efficiency:** The entire process must be executed within the real-time computational constraints of the NVIDIA Jetson Orin Nano platform.
- **Verification Protocol:** The agent must achieve a decisive victory in the "Ricercar" scenario when running on a Jetson Orin Nano, playing against the most difficult built-in "Petra" AI opponent set to the "Very Hard" difficulty level.

### v0.119: Metacognitive Retrospective and v0.120 Roadmap

- **Objective:** To demonstrate the highest possible level of self-awareness and recursive improvement by tasking the agent with analyzing its own learning process throughout the v0.11x cycle and proposing a data-driven plan for its future development.
- **Theoretical Foundation:** This is the ultimate expression of a Hofstadterian "Strange Loop" and Goodian RSI: the agent participating in the design of the next generation of

itself.[1] It is the final "jump out of the system"—the system of playing the game—into the meta-system of being a research project.

- **Architectural Modifications:** This version does not require new architecture but rather a new high-level operational mode for the existing Metacognitive Layer.
- **Implementation Details:** The agent's entire developmental history—the IEE's complete database of game traces, causal critiques, SMM modifications, performance metrics, and hardware profiles from v0.110 through v0.118—will be formatted and fed back into the agent's Capability Layer as a massive dataset. The agent will then be given a single, high-level prompt: "Analyze this dataset, which represents your own evolution. Identify the most impactful self-modifications, the most significant learning bottlenecks, and the remaining weaknesses in your cognitive architecture. Based on this analysis, propose a detailed, technically-grounded, and strategically-sound workplan for the v0.12x development cycle."
- **Verification Protocol:** The agent must produce a coherent, multi-page document that is technically plausible and formatted like this very workplan. The primary metric for success is human expert evaluation. The generated plan must be judged by the Project Prometheus leadership team as being strategically sound and, crucially, must contain at least one valid, non-obvious insight into its own cognitive architecture or learning process that the human designers had not previously considered. This would provide definitive proof of emergent, metacognitive intelligence.

# Section 3: Hardware Deployment and Performance Optimization Strategy

This section details the practical engineering plan for managing the dual-platform environment, which is a core constraint and a strategic opportunity for the project. The bifurcation of development and deployment platforms is a deliberate choice designed to maximize both creative exploration and real-world performance.

## Google Colab as the "Development & Evolution Sandbox"

- **Purpose:** The Google Colab environment will serve as the project's primary platform for all computationally unbounded tasks. Its dynamic access to a range of high-end NVIDIA GPUs (e.g., T4, P100, V100, A100), with significant VRAM (16-40 GB) and scalable CPU/RAM resources, is essential for the most computationally demanding phases of the RSI loop.[3] These tasks include the initial training and fine-tuning of the foundation

models, and more critically, the computationally explosive evolutionary and analogical searches performed by the SMM.
- **Workflow:** The SMM's "genetic programming" routines (v0.111) and the "Analogical Search Engine" (v0.116) will execute exclusively within the Colab environment. When the SMM generates a new candidate agent architecture or strategy (a new "generation"), it will be instantiated, trained, and validated within a simulated *0 A.D.* environment running inside the Colab instance. This allows for broad, unconstrained exploration of the solution space without being limited by real-time performance requirements.

## NVIDIA Jetson Orin Nano as the "Real-World Deployment Target"

- **Purpose:** The NVIDIA Jetson Orin Nano (specifically the 8GB model) serves as the project's resource-constrained, real-time inference environment.[7] It is the "ground truth" for performance. All verification protocols that involve real-time gameplay and efficiency measurements (v0.112, v0.117, v0.118) will be executed on this platform. Its specific performance characteristics—up to 40 INT8 TOPS of AI performance, a 1024-core Ampere architecture GPU, and 8GB of LPDDR5 memory—define the physical reality that the agent must learn to optimize for.[37]
- **Optimization Pipeline:** A critical component of the project's engineering infrastructure is the "Colab-to-Jetson Pipeline." This is an automated workflow that takes a new agent generation validated in Colab and rigorously optimizes it for deployment on the Jetson. This process will involve several key steps:
  1. **Model Quantization:** Leveraging libraries like NVIDIA TensorRT-LLM to convert the model's neural network weights from 32-bit or 16-bit floating-point (FP32/FP16) to 8-bit integer (INT8) precision. This drastically reduces the memory footprint and computational cost, allowing larger models to run and leveraging the Orin's dedicated Tensor Cores for maximum throughput.[35]
  2. **Kernel Fusion:** Using the TensorRT compiler to analyze the computational graph of the model and fuse multiple sequential operations (e.g., a convolution followed by an activation function) into a single, highly optimized GPU kernel, reducing memory access overhead.
  3. **Power Mode Management:** The agent's deployment script will be designed to programmatically set the Jetson's power mode (e.g., the 7W low-power mode or the 15W high-performance mode) based on the current strategic context or task requirements, making power management an explicit part of the agent's operational strategy.[37]

## The Hardware-in-the-Loop Feedback Cycle

The dual-platform strategy is not a one-way street. The performance data gathered by the IEE's profiling module on the Jetson—including latency, FPS, memory usage, and power draw—will be logged and transmitted back to the Metacognitive Layer running in Colab. This data becomes a critical input for the next round of self-improvement. For the tasks in v0.117 in particular, this hardware performance data becomes a primary component of the reward signal, creating a true hardware-in-the-loop RSI cycle. The agent in Colab is thus not just learning to win at *0 A.D.*; it is learning to generate solutions that are efficient and elegant on its target physical "body."

## Table 2: Hardware Capability and Optimization Plan

| Platform | Key Specifications | Primary Role in Project | Key Software/Optimization | Targeted Versions |
|---|---|---|---|---|
| **Google Colab** | **GPU:** NVIDIA T4/A100 (dynamic) **VRAM:** 16-40 GB **CPU:** Intel Xeon (2+ vCPUs) **RAM:** 13-96 GB | - Initial model training/fine-tuning - SMM Evolutionary Search - SMM Analogical Search Engine - IEE Causal Analysis | - PyTorch, TensorFlow, Keras 40<br>- OpenAI Gym, RLlib 41<br>- Pyvirtualdisplay for headless rendering 43<br>- Multi-agent RL frameworks 44 | v0.111, v0.113, v0.114, v0.116, v0.119 |
| **NVIDIA Jetson Orin Nano (8GB)** | **GPU:** 1024-core Ampere @ 625MHz **AI Perf:** 40 INT8 | - Real-time inference agent - Performance benchmarking | - NVIDIA JetPack SDK 8<br>- TensorRT-LLM for | v0.110, v0.112, v0.115, v0.117, v0.118 |

| | | | | |
|---|---|---|---|---|
| | TOPS **CPU:** 6-core Arm A78AE **RAM:** 8GB LPDDR5 (68 GB/s) **Power:** 7W-15W modes | - Hardware-in-the-loop RSI target - Final system validation | optimization 35 - INT8/FP16 Quantization - jetson-containers for deployment 35 | |

**Works cited**


1. Project Prometheus_ A Work Plan and Strategic Assessment for AGI Benchmarking in Complex Games.pdf
2. Hofstadter's Gödel, Escher, Bach: An Eternal Golden Braid Review— Choosing an operating model under uncertainty, incompleteness, and scale. | by Adnan Masood, PhD. | Aug, 2025 | Medium, accessed on October 6, 2025, https://medium.com/@adnanmasood/hofstadters-g%C3%B6del-escher-bach-an-eternal-golden-braid-choosing-an-operating-model-under-df35a183372e
3. GPU machine types | Compute Engine - Google Cloud, accessed on October 6, 2025, https://cloud.google.com/compute/docs/gpus
4. GPU Architecture - Google Colab, accessed on October 6, 2025, https://colab.research.google.com/github/d2l-ai/d2l-tvm-colab/blob/master/chapter_gpu_schedules/arch.ipynb
5. What's the Hardware Spec for Google Colaboratory | Saturn Cloud Blog, accessed on October 6, 2025, https://saturncloud.io/blog/whats-the-hardware-spec-for-google-colaboratory/
6. Jetson Benchmarks - NVIDIA Developer, accessed on October 6, 2025, https://developer.nvidia.com/embedded/jetson-benchmarks
7. NVIDIA Jetson Orin Nano Super Kit Elevates AI Performance - Tanna TechBiz, accessed on October 6, 2025, https://tannatechbiz.com/blog/post/nvidia-jetson-orin-nano-super-developer-kit
8. NVIDIA Jetson Orin Nano Developer Kit Gets a “Super” Boost | NVIDIA Technical Blog, accessed on October 6, 2025, https://developer.nvidia.com/blog/nvidia-jetson-orin-nano-developer-kit-gets-a-super-boost/
9. Good-I.-J.-1966.-Speculations-Concerning-the-First-Ultraintelligent-Machine.pdf, accessed on October 6, 2025, https://blog.biocomm.ai/wp-content/uploads/2023/06/Good-I.-J.-1966.-Speculations-Concerning-the-First-Ultraintelligent-Machine.pdf
10. About Causal Assessment | US EPA, accessed on October 6, 2025, https://www.epa.gov/caddis/about-causal-assessment
11. An Introduction to Causal Reinforcement Learning - Elias Bareinboim, accessed on October 6, 2025, https://causalai.net/r65.pdf
12. How does the brain convey meaning, accessed on October 6, 2025,

https://psychlab1.hanover.edu/Vision/Background/isomorphism.html
13. Notes on GEB - Computer Science, accessed on October 6, 2025, https://cs.lmu.edu/~ray/notes/geb/
14. Machine Learning in 0 AD - General Discussion - Wildfire Games Community Forums, accessed on October 6, 2025, https://wildfiregames.com/forum/topic/27617-machine-learning-in-0-ad/
15. Analogy as the Core of Cognition, accessed on October 6, 2025, https://worrydream.com/refs/Hofstadter_2001_-_Analogy_as_the_Core_of_Cognition.pdf
16. Good , X. 3. - Stacks, accessed on October 6, 2025, https://stacks.stanford.edu/file/gz727rg3869/gz727rg3869.pdf
17. Gwern.net - The Scientist Speculates, accessed on October 6, 2025, https://gwern.net/doc/science/1962-good-thescientistspeculates.pdf
18. Causal Interpretation of Self-Attention in Pre-Trained Transformers, accessed on October 6, 2025, https://proceedings.neurips.cc/paper_files/paper/2023/file/642a321fba8a0f03765318e629cb93ea-Paper-Conference.pdf
19. libo-huang/Awesome-Causal-Reinforcement-Learning - GitHub, accessed on October 6, 2025, https://github.com/libo-huang/Awesome-Causal-Reinforcement-Learning
20. Causal Reinforcement Learning, accessed on October 6, 2025, https://crl.causalai.net/
21. Causal Reinforcement Learning: A Survey - OpenReview, accessed on October 6, 2025, https://openreview.net/pdf?id=qqnttX9LPo
22. Reinforcement Learning-Based Nonlinear Model Predictive Controller for a Jacketed Reactor: A Machine Learning Concept Validation Using Jetson Orin | ACS Omega - ACS Publications, accessed on October 6, 2025, https://pubs.acs.org/doi/10.1021/acsomega.5c03219
23. 16_Reinforcement_Learning.ipynb - Colab - Google, accessed on October 6, 2025, https://colab.research.google.com/github/Hvass-Labs/TensorFlow-Tutorials/blob/master/16_Reinforcement_Learning.ipynb
24. Interface - Gradio Docs, accessed on October 6, 2025, https://www.gradio.app/docs/gradio/interface
25. The Man Who Would Teach Machines to Think - The Atlantic - Computer Science, accessed on October 6, 2025, http://cs.oswego.edu/~jhundal/other/AtlanticArticleOnHofstadter.pdf
26. Can GPT-3 Make Analogies?. By Melanie Mitchell - Medium, accessed on October 6, 2025, https://medium.com/@melaniemitchell.me/can-gpt-3-make-analogies-16436605c446
27. The Copycat project: A model of mental fluidity and analogy-making - Percepts and Concepts Lab, accessed on October 6, 2025, https://pcl.sitehost.iu.edu/rgoldsto/courses/concepts/copycat.pdf
28. Tite Copycat Project: A Model of Mental Fluidity - and Analogy-making - Putnam

Program, accessed on October 6, 2025, https://theputnamprogram.files.wordpress.com/2012/03/dhfa_chp5.pdf
29. The Copycat Project: An Experiment in Nondeterminism and Creative Analogies, accessed on October 6, 2025, https://www.semanticscholar.org/paper/The-Copycat-Project%3A-An-Experiment-in-and-Creative-Hofstadter/27b7eb6239ea48386e2fbcd71faf858f6c5e5637
30. (PDF) The copycat project: A model of mental fluidity and analogy-making - ResearchGate, accessed on October 6, 2025, https://www.researchgate.net/publication/243649607_The_copycat_project_A_model_of_mental_fluidity_and_analogy-making
31. The Copycat Project: An Experiment in Nondeterminism and Creative Analogies. - DTIC, accessed on October 6, 2025, https://apps.dtic.mil/sti/tr/pdf/ADA142744.pdf
32. Copycat Architecture - Science@SLC, accessed on October 6, 2025, https://science.slc.edu/jmarshall/courses/2005/fall/cs151/lectures/copycat/
33. Recursive Self-Improvement - LessWrong, accessed on October 6, 2025, https://www.lesswrong.com/w/recursive-self-improvement
34. The Problem - Machine Intelligence Research Institute, accessed on October 6, 2025, https://intelligence.org/the-problem/
35. Running LLMs with TensorRT-LLM on NVIDIA Jetson Orin Nano Super - Collabnix, accessed on October 6, 2025, https://collabnix.com/running-llms-with-tensorrt-llm-on-nvidia-jetson-orin-nano-super/
36. Google Colab, accessed on October 6, 2025, https://research.google.com/colaboratory/faq.html
37. Solving Entry-Level Edge AI Challenges with NVIDIA Jetson Orin Nano, accessed on October 6, 2025, https://developer.nvidia.com/blog/solving-entry-level-edge-ai-challenges-with-nvidia-jetson-orin-nano/
38. Large Language Models on Small Resource-Constrained Systems: Performance Analysis and Trade-offs - arXiv, accessed on October 6, 2025, https://arxiv.org/html/2412.15352v1
39. Maximizing Deep Learning Performance on NVIDIA Jetson Orin with DLA, accessed on October 6, 2025, https://developer.nvidia.com/blog/maximizing-deep-learning-performance-on-nvidia-jetson-orin-with-dla/
40. Keras: Deep Learning for humans, accessed on October 6, 2025, https://keras.io/
41. Gym Documentation, accessed on October 6, 2025, https://www.gymlibrary.dev/
42. Intro to Reinforcement Learning | OpenAI Gym, RLlib & Google Colab - Anyscale, accessed on October 6, 2025, https://www.anyscale.com/blog/an-introduction-to-reinforcement-learning-with-openai-gym-rllib-and-google
43. ajit2704/Reinforcement-Learning-on-google-colab - GitHub, accessed on October 6, 2025, https://github.com/ajit2704/Reinforcement-Learning-on-google-colab
44. [2509.03682] A Comprehensive Review of Multi-Agent Reinforcement Learning in

Video Games - arXiv, accessed on October 6, 2025,
https://arxiv.org/abs/2509.03682